

Projeto: ForkUp

Grupo 88:

Nome: Luís Felipe Garcia Menezes - **RM:** 371055

1. Introdução

Descrição do Problema

Um grupo de restaurantes da região decidiu desenvolver um sistema de gestão compartilhado para reduzir custos com soluções individuais. O objetivo é criar uma plataforma única que permita aos restaurantes gerenciar suas operações e aos clientes consultar informações, avaliar estabelecimentos e realizar pedidos online. Devido a limitações financeiras, o sistema será desenvolvido em fases, possibilitando uma implementação gradual, com melhorias contínuas baseadas no uso e feedback dos usuários.

Objetivo do Projeto

Desenvolver uma API REST robusta e escalável utilizando Spring Boot, responsável pelo gerenciamento de usuários e endereços, com foco em boas práticas de arquitetura, validação de dados, segurança (criptografia de senhas) e persistência em banco de dados relacional. A solução deve atender aos requisitos funcionais do sistema e servir como base para futuras evoluções.

2. Arquitetura do Sistema

A aplicação **ForkUp** foi projetada seguindo os princípios de **arquitetura em camadas (Layered Architecture)**, amplamente utilizada em aplicações corporativas com Spring Boot, com foco em **separação de responsabilidades, manutenibilidade e escalabilidade**.

A solução é executada em ambiente containerizado utilizando Docker, com dois serviços principais: a aplicação backend (Spring Boot) e o banco de dados PostgreSQL, que se comunicam através de uma rede interna.

O fluxo de uma requisição segue o padrão:

Cliente → Controller → Service → Repository → Banco de Dados

A resposta percorre o caminho inverso, sendo convertida em DTO antes de ser retornada ao cliente.

2.1 Camadas da Aplicação

API / Controller

- Responsável por receber requisições HTTP e atuar como adaptador entre o cliente e a aplicação.
- Realiza validações básicas de entrada com Jakarta Validation (@Valid, @Validated).
- Expõe endpoints versionados (/api/v1/* e /api/v2/*).

Service (Business Layer)

- Contém as regras de negócio da aplicação.
- Orquestra chamadas entre repositórios e outros componentes.
- Utiliza @Transactional para garantir consistência das operações.
- Lida com exceções de negócio específicas.

Repository (Data Access)

- Interfaces que estendem JpaRepository.

- Responsáveis pelo acesso ao banco de dados, incluindo operações CRUD, paginação e consultas customizadas.

Mapper

- Utiliza MapStruct para conversão entre entidades e DTOs/VOs.
- Garante desacoplamento entre as camadas e evita exposição direta das entidades.

Domain / Entity

- Representa o modelo de dados da aplicação.
- Utiliza anotações JPA (@Entity, @Table) para mapeamento com o banco de dados.

Exception Handling

- Implementa tratamento centralizado de erros com @ControllerAdvice.
- Utiliza o padrão Problem Details (RFC 7807) para respostas padronizadas de erro.

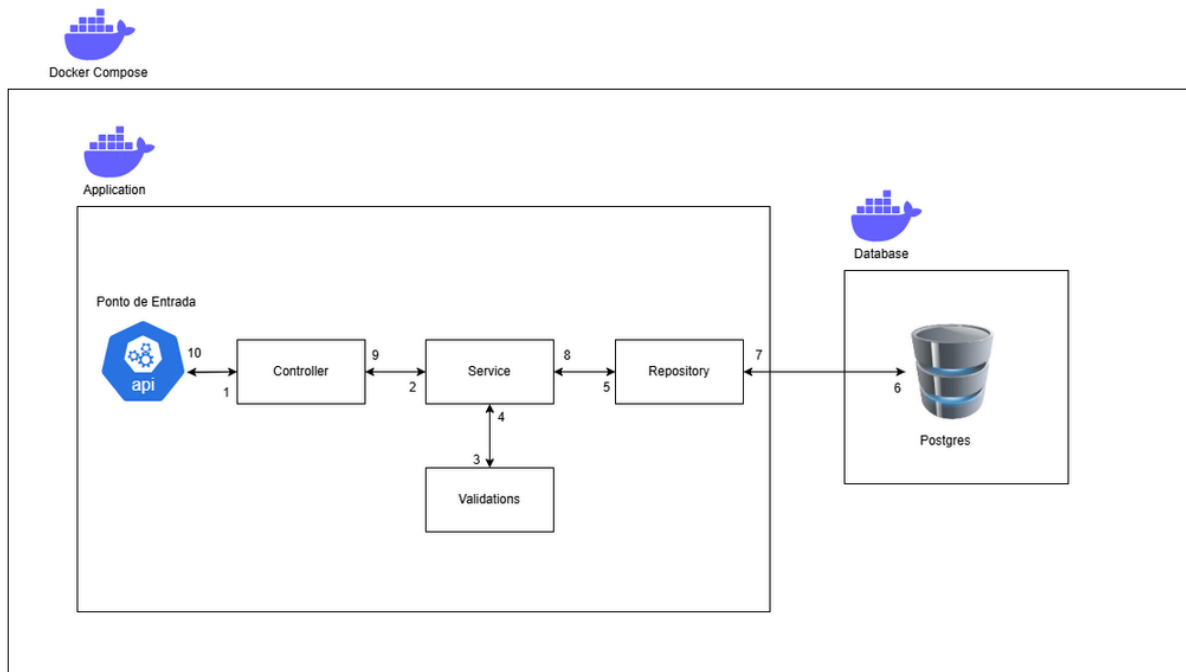
2.2 Containerização

A aplicação é executada utilizando Docker Compose, com dois serviços principais:

- **app** → Responsável por executar a API Spring Boot
- **postgres** → Responsável pela persistência de dados

Os serviços se comunicam através de uma rede interna do Docker, utilizando o nome do serviço como hostname (ex: `postgres`), e são configurados por meio de variáveis de ambiente definidas no arquivo `.env`.

2.3 Diagrama da Arquitetura



3. Modelagem das Entidades e Relacionamentos

A modelagem do sistema foi baseada em um modelo relacional, com foco na organização e integridade das informações.

3.1 Entidades principais:

- **Usuário**
 - Representa os clientes do sistema
 - Contém informações como nome, email, login e senha
 - A senha é armazenada de forma criptografada
- **Endereço**
 - Representa os endereços associados a um usuário
 - Contém dados como rua, número, cidade, estado e CEP

3.2 Uso de Enums e Converters

O sistema utiliza enumerações para representar valores fixos e controlados, como **status do usuário** e **tipo de usuário**, por meio das classes StatusEnum e TipoUsuarioEnum.

Para garantir uma melhor integração com o banco de dados e evitar o uso direto de strings ou valores numéricos nas entidades, foram implementados **conversores JPA (AttributeConverter)**.

Esses conversores são responsáveis por:

- Converter automaticamente os valores do banco de dados para enums na aplicação
- Converter enums para seus respectivos valores persistidos no banco.

3.3 Relacionamentos

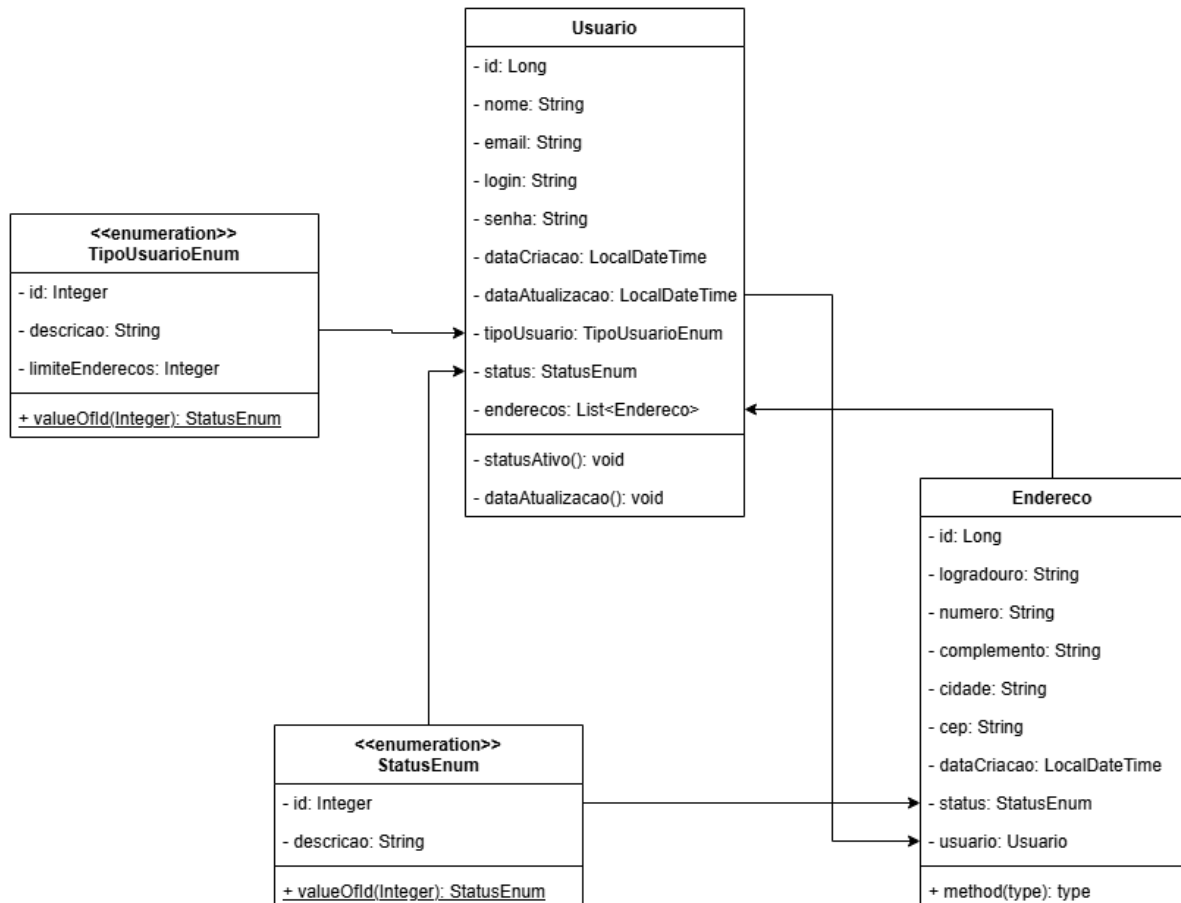
- Um **Usuário** pode possuir um ou mais **Endereços**
- Um **Endereço** pertence a apenas um **Usuário**

3.4 Regras de negócio refletidas

- Dependendo do tipo de usuário, pode haver limite de endereços
- Não é permitido cadastrar usuários com email ou login duplicados
- Endereços estão sempre vinculados a um usuário válido

3.5 Diagrama

Abaixo está o diagrama de entidades e relacionamentos do sistema:



4. Descrição dos Endpoints

4.1 Tabela de Endpoints

Endpoint	Método	Descrição
/api/v1/auth/login	POST	Realiza autenticação do usuário

/api/v1/usuarios	GET	Lista usuários com paginação
/api/v1/usuarios/{id}	GET	Busca usuário por ID
/api/v1/usuarios	POST	Cria um novo usuário
/api/v1/usuarios/{id}	PUT	Atualiza dados do usuário
/api/v1/usuarios/{id}/senha	PUT	Atualiza a senha do usuário
/api/v1/usuarios/{id}	DELETE	Remove usuário
/api/v1/enderecos/usuario/{idUsuario}	GET	Lista endereços de um usuário
/api/v1/enderecos/{id}	GET	Busca endereço por ID
/api/v1/enderecos	POST	Cria um novo endereço
/api/v1/enderecos/{id}	PUT	Atualiza dados do endereço
/api/v1/enderecos/{id}	DELETE	Remove endereço

4.2 Exemplos de Requisição e resposta

Autenticação

Login

POST - /api/v1/auth/login

Request:

```
{
  "login": "joao_silva",
  "senha": "SenhaSegura!123"
}
```

Reponse (200):

```
{  
  "mensagem": "Login realizado com sucesso"  
}
```

Usuários

Buscar usuário por ID

GET - /api/v1/usuarios/{id}

Response (200):

```
{  
  "id": 1,  
  "nome": "Dono de Restaurante da Silva",  
  "email": "dono.restaurante@gmail.com",  
  "login": "donoRestaurante23",  
  "tipoUsuario": "DONO_RESTAURANTE",  
  "status": "ATIVO",  
  "dataCriacao": "2026-05-05T02:33:31.021191"  
}
```

Endereços

Criar Endereço

POST - /api/v1/enderecos

Request:

```
{  
  "logradouro": "Rua das Flores",  
  "numero": "123",  
  "complemento": "Apto 45",  
  "cidade": "São Paulo",  
  "cep": "12345-678",  
  "idUsuario": 1  
}
```

Response (201):


```
{  
    "id" : 10  
}
```

5. Documentação Swagger

5.1 Documentação da API com Swagger

A API do sistema **ForkUp** possui documentação interativa gerada automaticamente utilizando o **SpringDoc OpenAPI (Swagger)**.

Essa documentação permite visualizar e testar todos os endpoints disponíveis de forma simples e intuitiva, sem a necessidade de ferramentas externas.

A documentação pode ser acessada após a execução da aplicação:

<http://localhost:3710/swagger-ui.html>

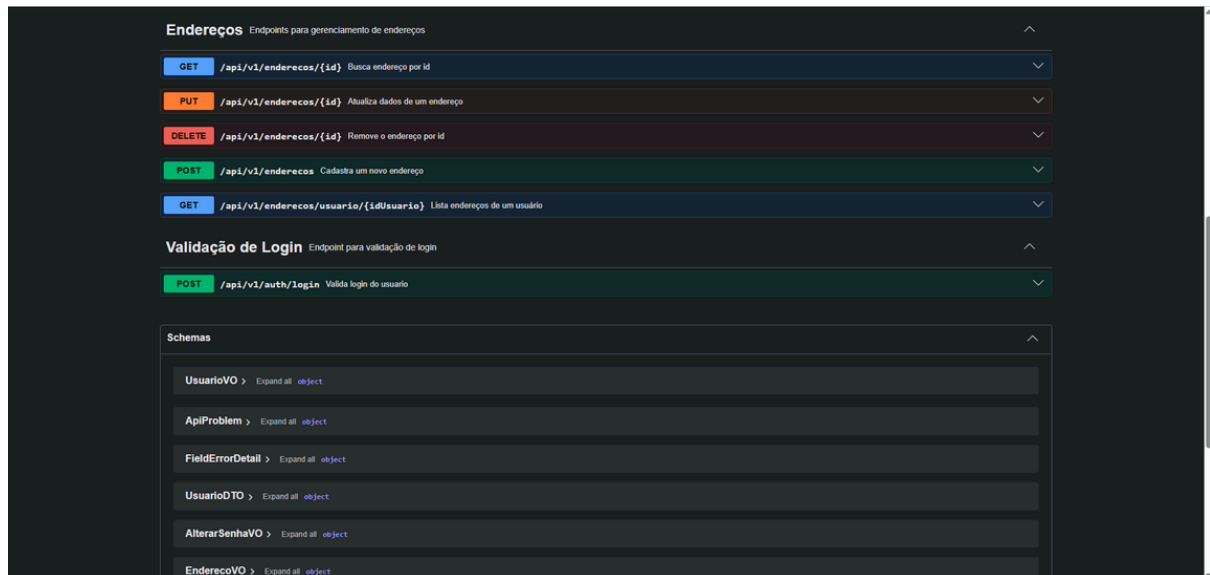
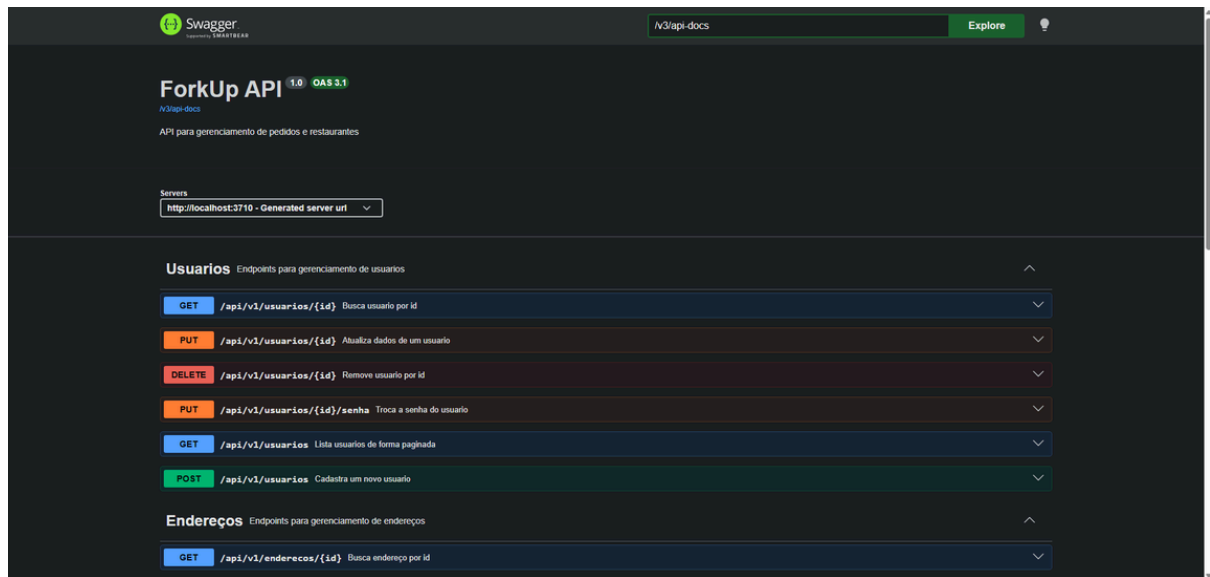
O Swagger disponibiliza:

- Listagem completa dos endpoints da API
- Descrição detalhada de cada operação (GET, POST, PUT, DELETE)
- Visualização dos parâmetros de entrada
- Exemplos de requisição e resposta
- Possibilidade de executar requisições diretamente pela interface ("Try it out")

5.2 Exemplos de Interface

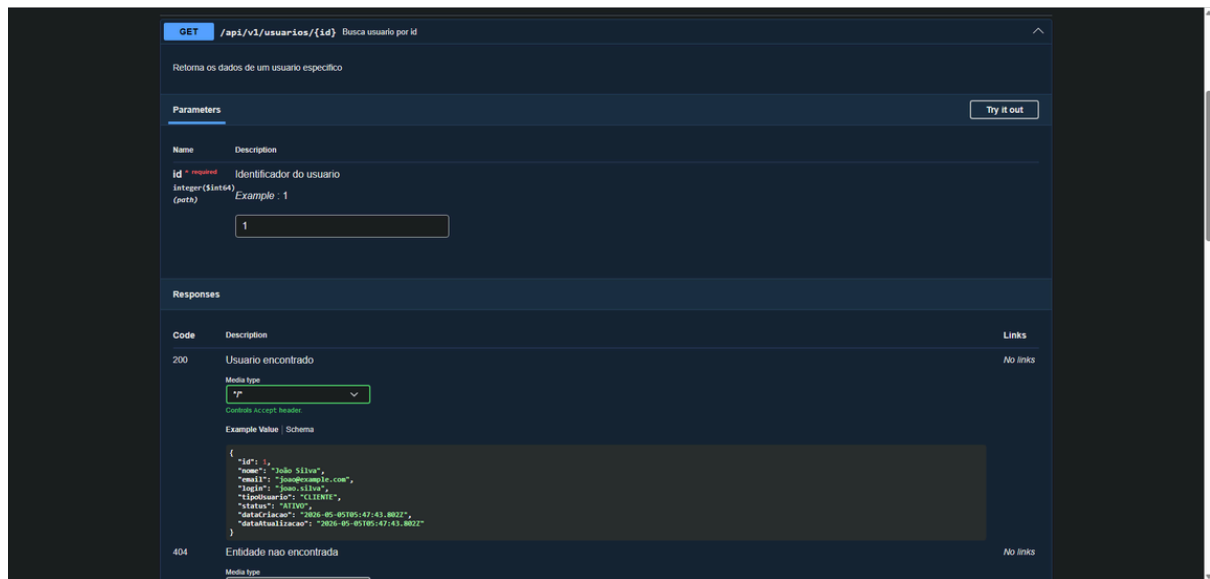
Tela Inicial

A tela inicial apresenta todos os controllers e endpoints organizados por categoria.



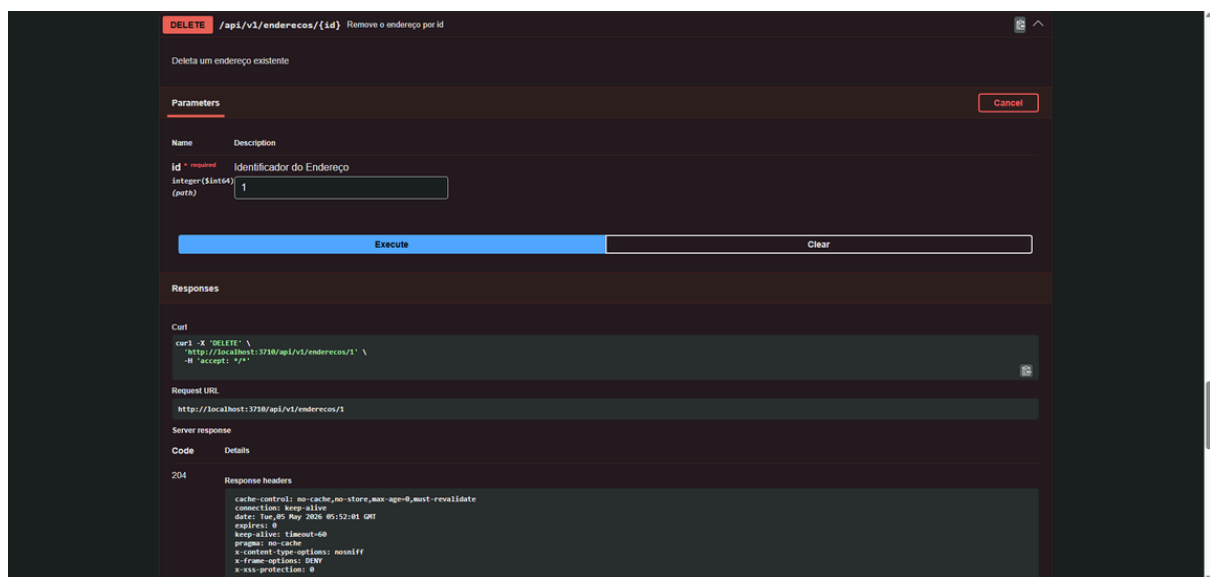
Exemplo de Endpoint Expandido

Ao expandir um endpoint, é possível visualizar detalhes da requisição, incluindo parâmetros, corpo da requisição e exemplos de resposta.



Execução de requisição

A funcionalidade "Try it out" permite testar os endpoints diretamente pela interface.



6. Documentação Postman

6.1 Testes da API com Postman

Para facilitar os testes e validações dos endpoints, foi disponibilizada uma **collection do Postman** contendo requisições prontas para todos os principais fluxos da aplicação.

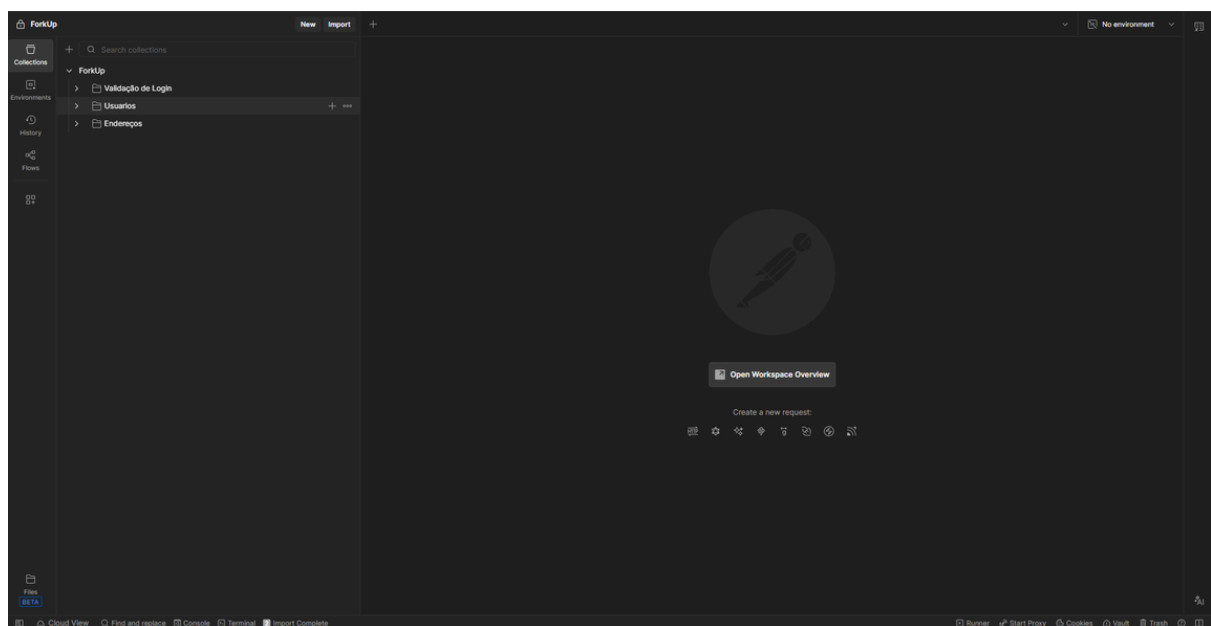
Arquivo da Collection

- Nome: ForkUp_postman_collection.json
- Localização: raiz do projeto

6.2 Exemplos da Interface

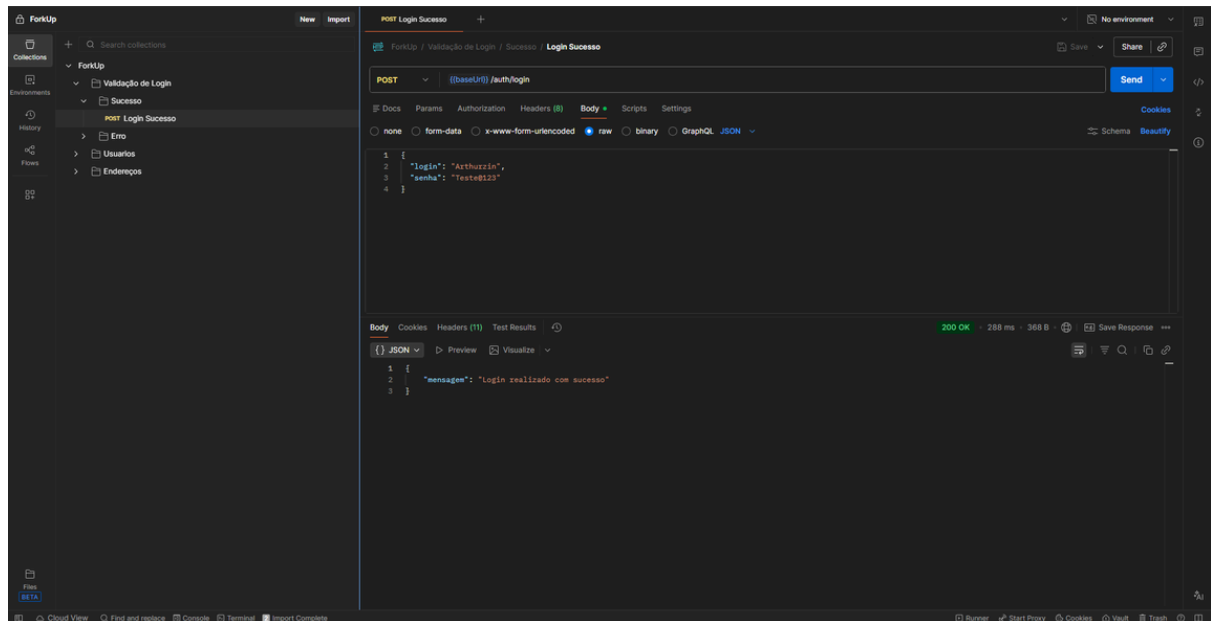
Collection importada

Visualização da collection com todos os endpoints organizados por categoria.



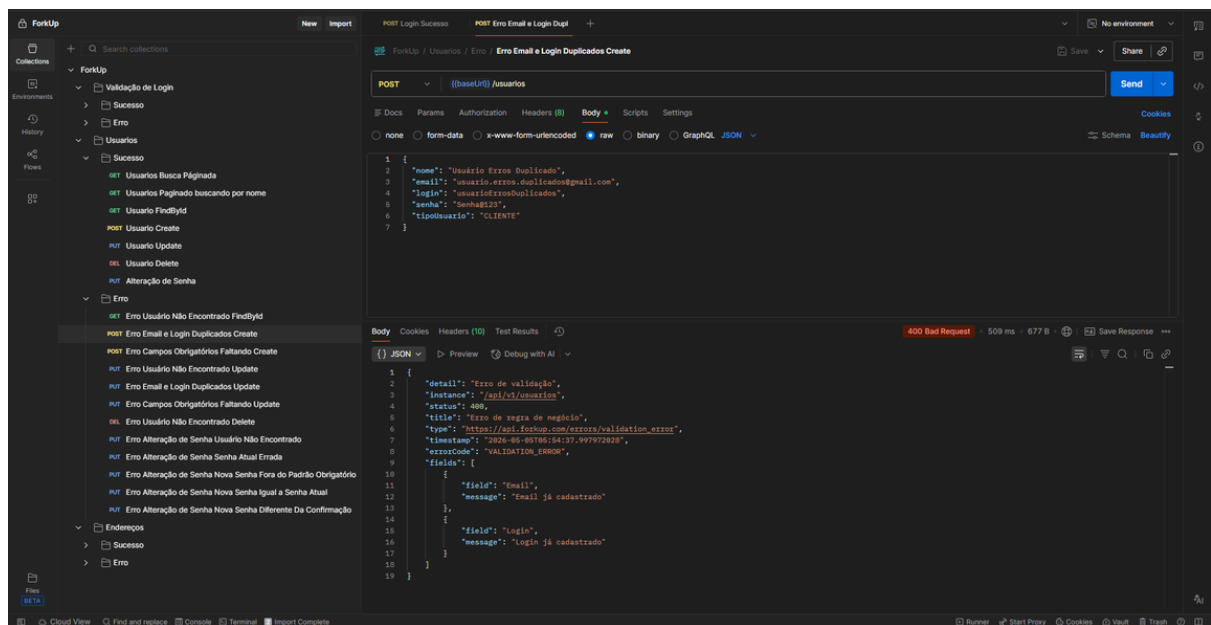
Execução de requisição

Exemplo de execução de uma requisição com retorno da API.



Separação em Sucesso e Error

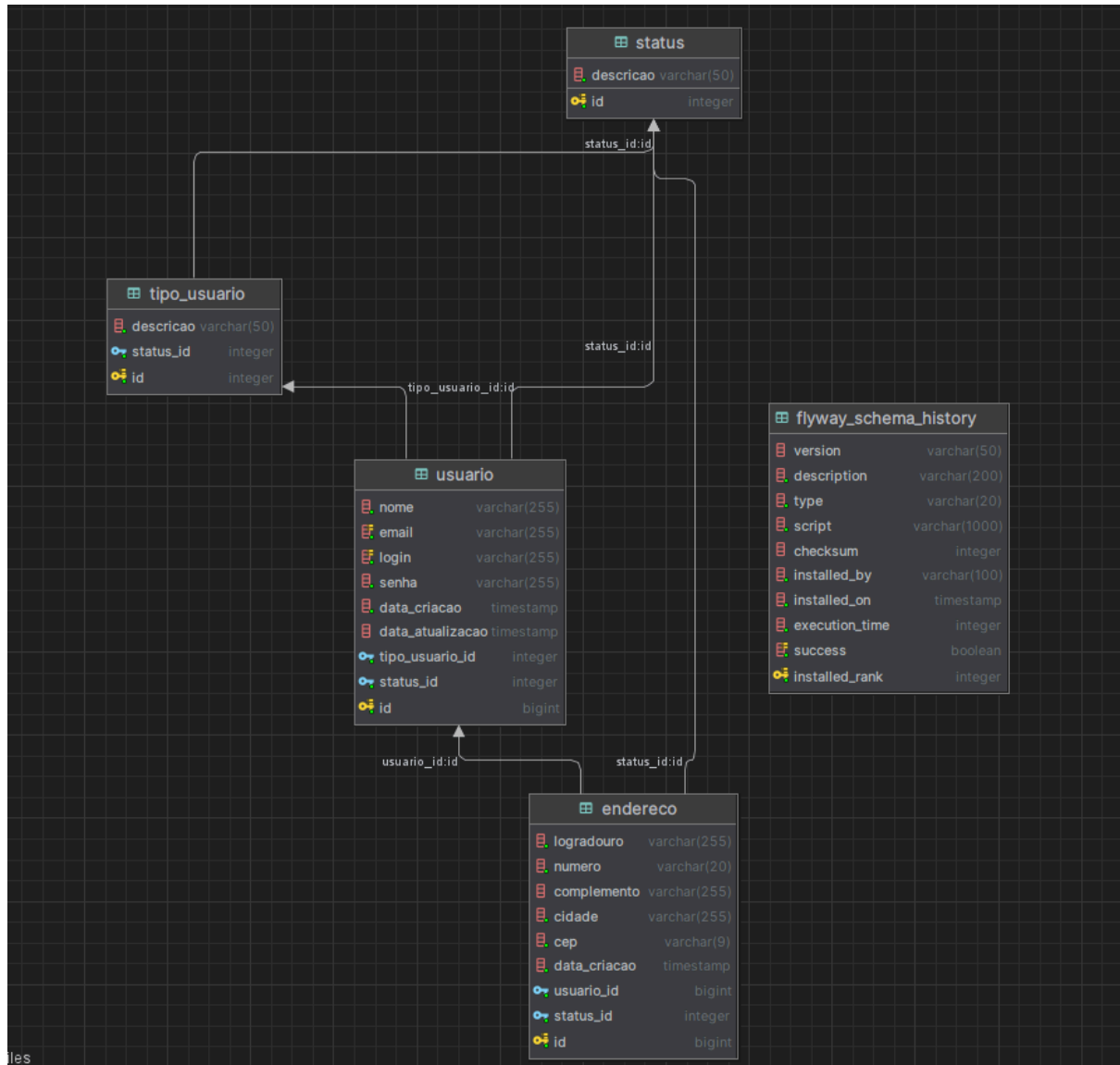
As requisições estão organizadas por domínio (Usuários, Endereços, Autenticação) e para cada um possuem pastas, com massas de testes prontas para as execuções de sucesso e error



7. Estrutura do Banco de Dados

O banco de dados do sistema **ForkUp** foi modelado utilizando o modelo relacional, com foco na integridade dos dados e no suporte às regras de negócio da aplicação.

Atualmente, o sistema é composto pelas seguintes tabelas:



8. Execução da Aplicação com Docker Compose

8.1 Clonar o repositório

```
git clone https://github.com/LuisFelipeGM/ForkUp
cd ForkUp
```

8.2 Configurar variáveis de ambiente

O projeto utiliza variáveis de ambiente para configuração do banco de dados e da aplicação. Criar arquivo .env:

Copie o arquivo de exemplo:

```
cp .env.example .env
```

No Windows (PowerShell):

```
copy .env.example .env
```

Editar o .env

```
DB_NAME=forkup_db
```

```
DB_USER=admin
```

```
DB_PASSWORD=sua_senha
```

```
DB_PORT=5432
```

```
APP_PORT=3710
```

Explicação das variáveis:

Variável	Descrição
DB_NAME	Nome do banco de dados

DB_USER	Usuário do banco
DB_PASSWORD	Senha do banco
DB_PORT	Porta do PostgreSQL
APP_PORT	Porta da aplicação

8.3 Executar a aplicação

Para construir a imagem e subir os containers:

docker compose up --build -d

8.4 Acessar a aplicação

Após a inicialização:

- API: <http://localhost:3710>
- Swagger: <http://localhost:3710/swagger-ui.html>

Observações importantes

- O arquivo .env não é versionado por conter dados sensíveis
- Utilize o .env.example como base
- A aplicação se conecta ao banco utilizando o nome do serviço (postgres) como host

9. Repositório do Projeto

URL do Repositório

<https://github.com/LuisFelipeGM/ForkUp>